

7. Configuration Management

Report section prepared by **Member 4 — Md Sharior Omar Fahim (63909)**, GitHub & Configuration Management.

Section number 7 follows Member 3's Risk & Issue Management section (Section 6).

7.1 Configuration Management Overview

Configuration management (CM) is the discipline of tracking and controlling changes to project deliverables, documentation, and any other project artifacts throughout the project lifecycle. For the LMS project, CM ensures that all project documents, design files, and reports are stored securely, that changes are tracked with a full history of who changed what and when, and that team members can collaborate without overwriting one another's work.

The primary CM tool selected for this project is **GitHub**, a cloud-based platform built on the **Git** version-control system. GitHub provides repository hosting, commit history, branching, and collaboration features that together satisfy the configuration-management requirements of the project.

7.2 Configuration Management Process

The CM process followed in this project consists of the following steps:

1. **Repository Setup** — A central GitHub repository (`LMS-Project-Plan`) was created to host all project artifacts.

1. **Folder Organisation** — Deliverables were organised into member-specific folders under a `docs/` directory for traceability.

1. **Version Tracking** — Every change is recorded as a Git commit with a descriptive message, timestamp, and author identity, creating a complete audit trail.

1. **Branching Strategy** — Feature branches (for example, `feature/cm-documentation`) were used to develop documentation in isolation before merging into the `main` branch, preventing incomplete work from affecting the baseline.

1. **Change Control** — All changes to the repository are visible in the commit history. Any file modification, addition, or deletion is logged permanently.

1. **Backup and Redundancy** — The repository exists both locally (on team members'

machines) and remotely (on GitHub's servers), providing redundancy against data loss.

7.3 Repository Structure

```
LMS-Project-Plan/
├─ README.md
├─ .gitignore
├─ docs/
│   ├─ member1-introduction/
│   │   └─ Member1.pdf
│   ├─ member2-planning/
│   │   └─ Member2.pdf
│   ├─ member3-risk-management/
│   │   └─ Member3.docx
│   ├─ member4-configuration-management/
│   │   └─ configuration-management-section.md
│   └─ member5-final-report/
└─ screenshots/
    ├─ 01-repo-creation.png
    ├─ 02-folder-structure.png
    └─ ...
```

7.4 Version Control Evidence

Table 7.1 summarises the key commits made while building the repository. The full, authoritative history is available on GitHub under the repository's *Commits* tab, and can be reproduced locally with `git log --oneline --graph`.

Table 7.1 — Key commits

#	Date	Branch	Commit message	Files changed
1	30 May 2026	main	Initialize repository structure and .gitignore	.gitignore , folder placeholders
2	30 May 2026	main	Add member deliverables: Introduction, Planning, and Risk Management	Member1.pdf , Member2.pdf , Member3.docx
3	30 May 2026	main	Add project README with overview, team roles, and structure	README.md

#	Date	Branch	Commit message	Files changed
4	30 May 2026	feature/cm-documentation	Add configuration management section (Section 7) — initial draft	configuration-management-section.md
5	30 May 2026	feature/cm-documentation	Expand CM section with evidence table, tools, and benefits	configuration-management-section.md
6	30 May 2026	main	Merge branch 'feature/cm-documentation'	(merge)
7	30 May 2026	main	Add screenshots guide for version-control evidence	screenshots/README.md
8	30 May 2026	main	Update README to link the configuration management section	README.md

The repository also demonstrates **branching and merging**: the configuration-management documentation was developed on the `feature/cm-documentation` branch and then merged back into `main` with a non-fast-forward merge, so the branch history is preserved in the graph.

7.5 Tools Used for Configuration Management

Table 7.2 — CM tools

Tool	Purpose
Git	Local version control — tracking file changes, creating commits and branches
GitHub	Remote repository hosting — cloud storage, collaboration, and commit-history visualisation
Git Bash / Terminal	Command-line interface for executing Git commands

7.6 Benefits of Using GitHub for This Project

- **Traceability** — Every change is permanently logged with author, date, and description.
- **Collaboration** — Multiple team members can work simultaneously without conflicts.
- **Backup** — The repository is stored both locally and in the cloud.
- **Accountability** — Each commit is attributed to a specific team member.
- **Rollback capability** — Any previous version of any file can be restored if needed.

7.7 Screenshots

The screenshots listed below are stored in the repository's `screenshots/` folder and should be inserted here as figures. (See `screenshots/README.md` for the capture guide.)

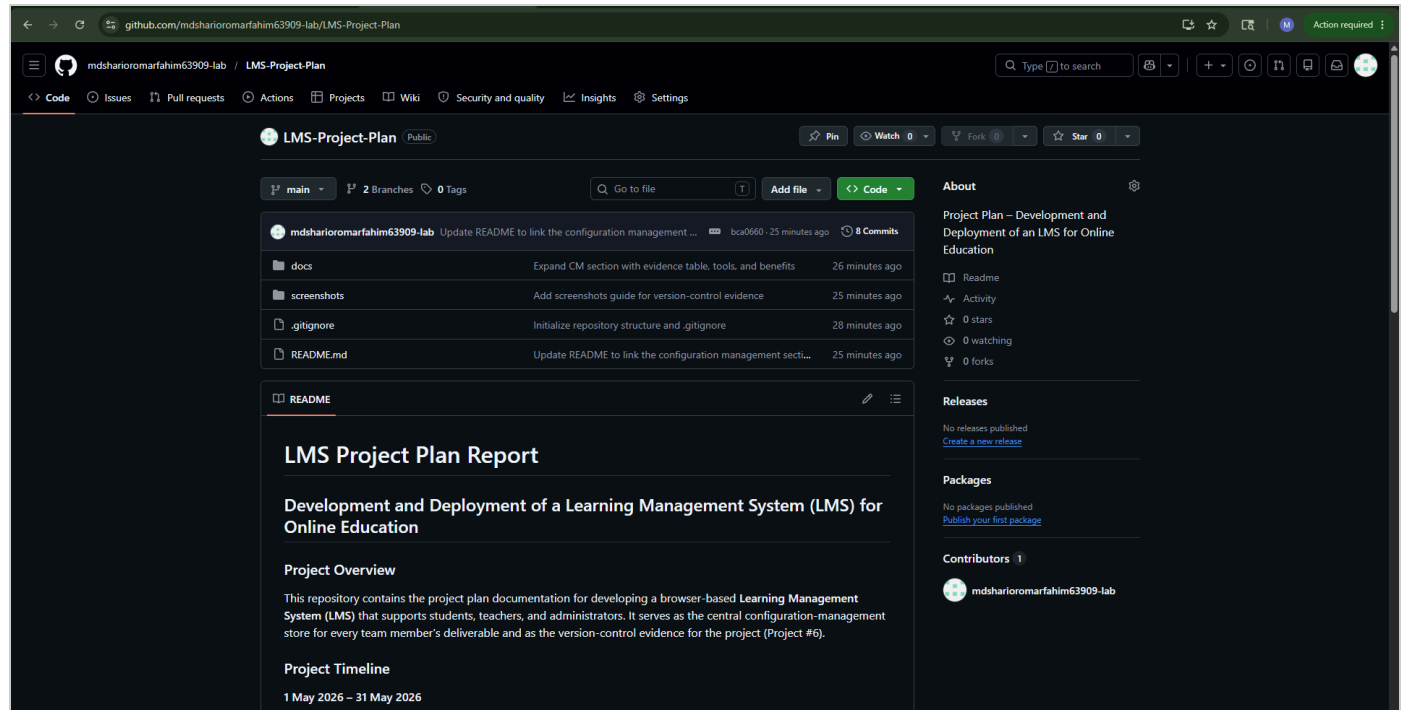


Figure 7.1 — GitHub repository main page

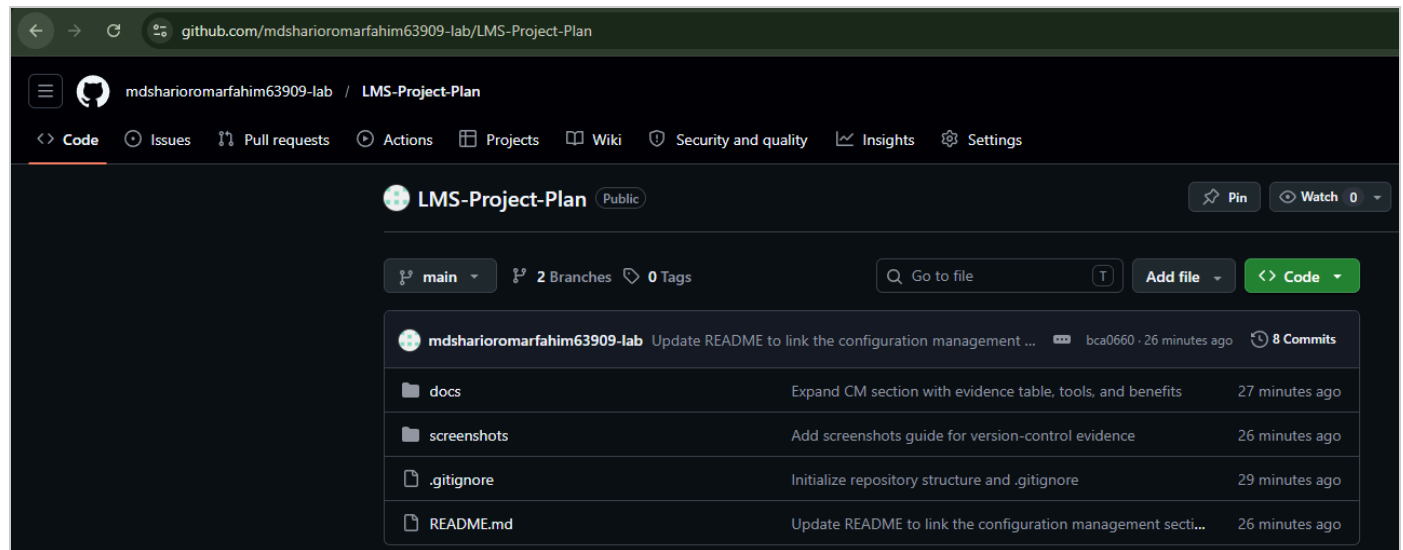


Figure 7.2 — Folder structure in the repository

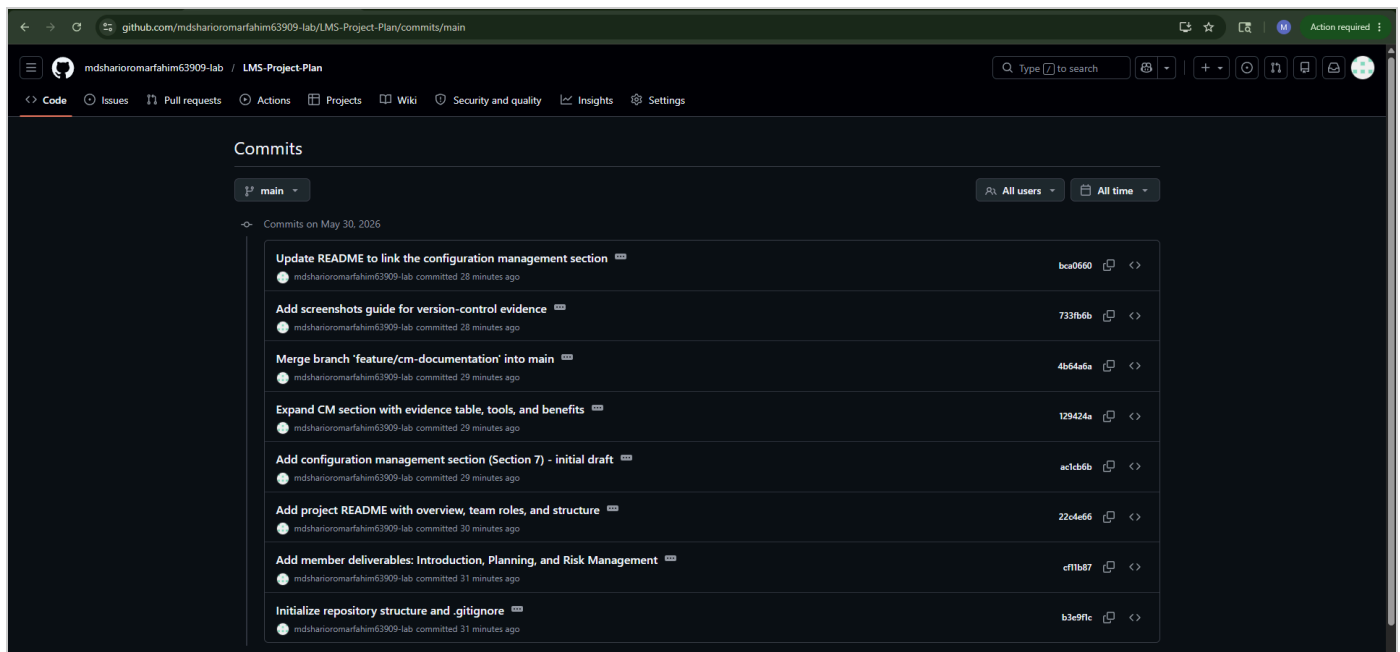


Figure 7.3 — Commit history showing multiple commits

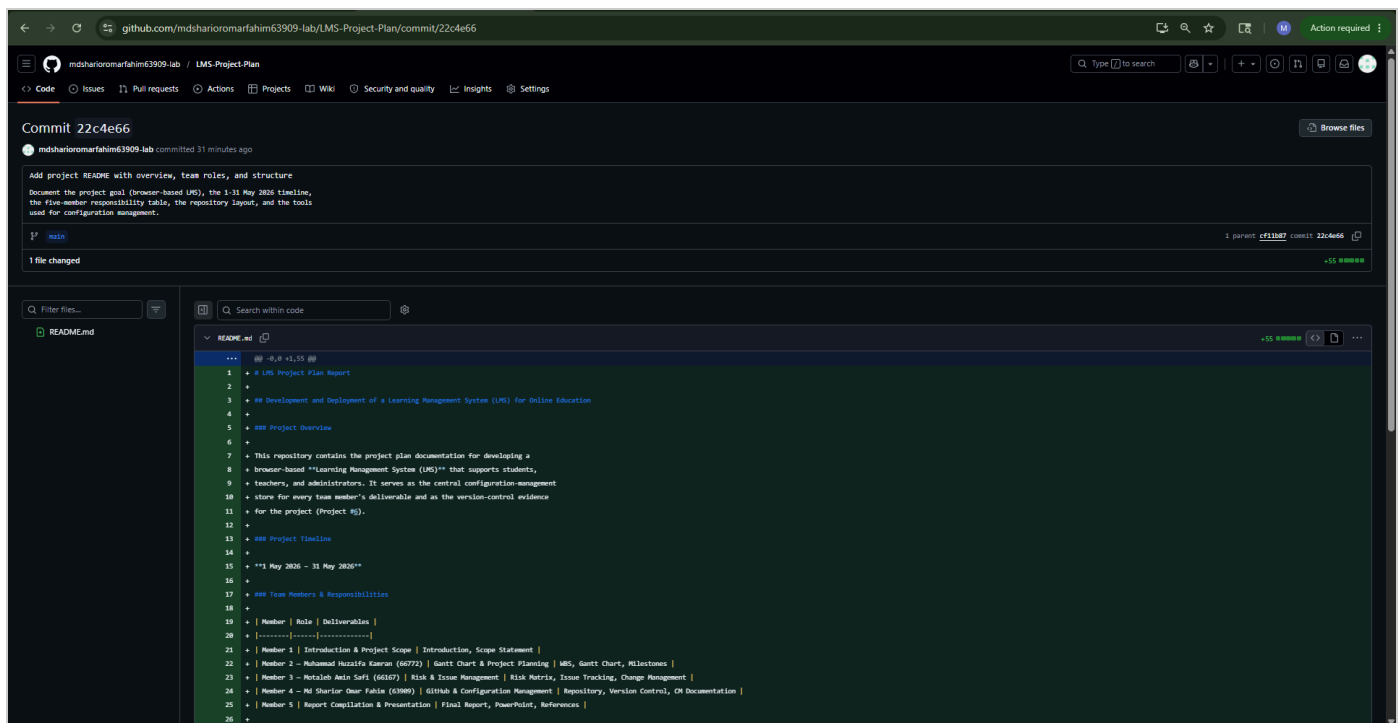


Figure 7.4 — Commit detail showing file changes / diff view

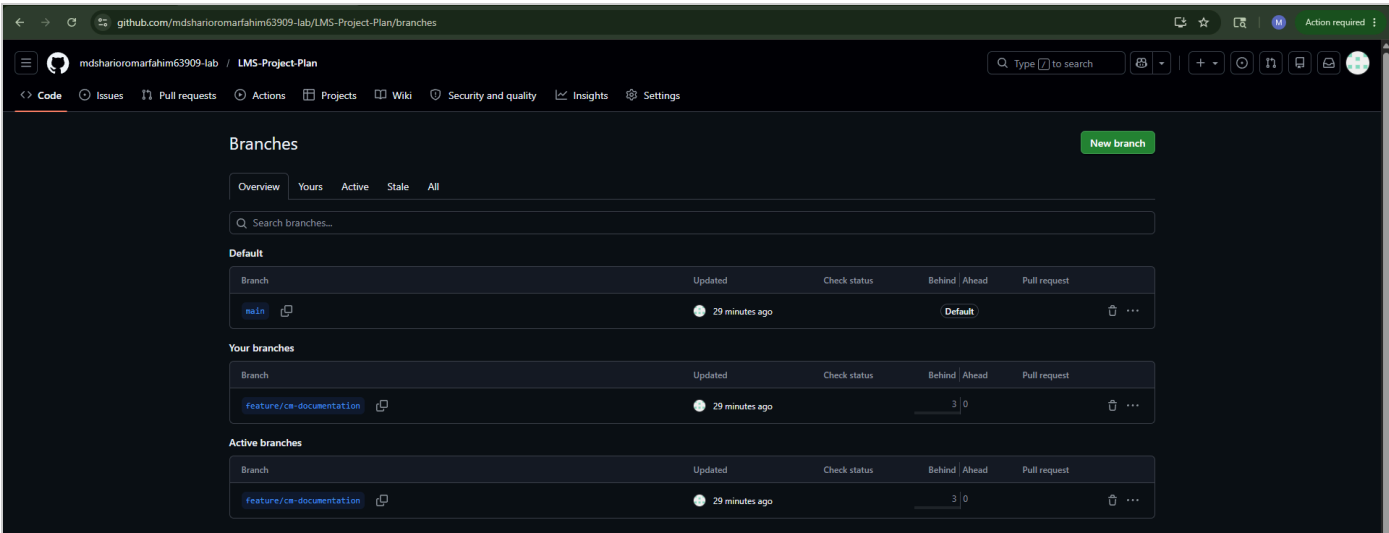


Figure 7.5 — Branch usage evidence

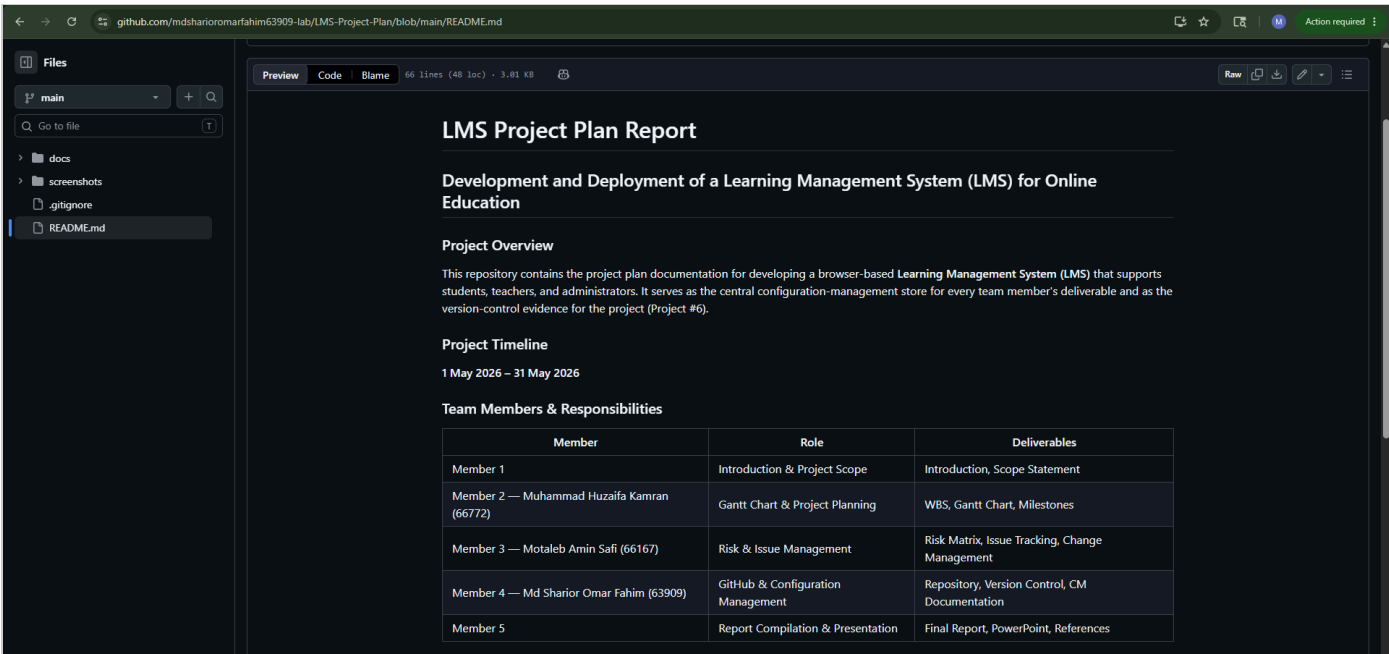


Figure 7.6 — README rendered on GitHub

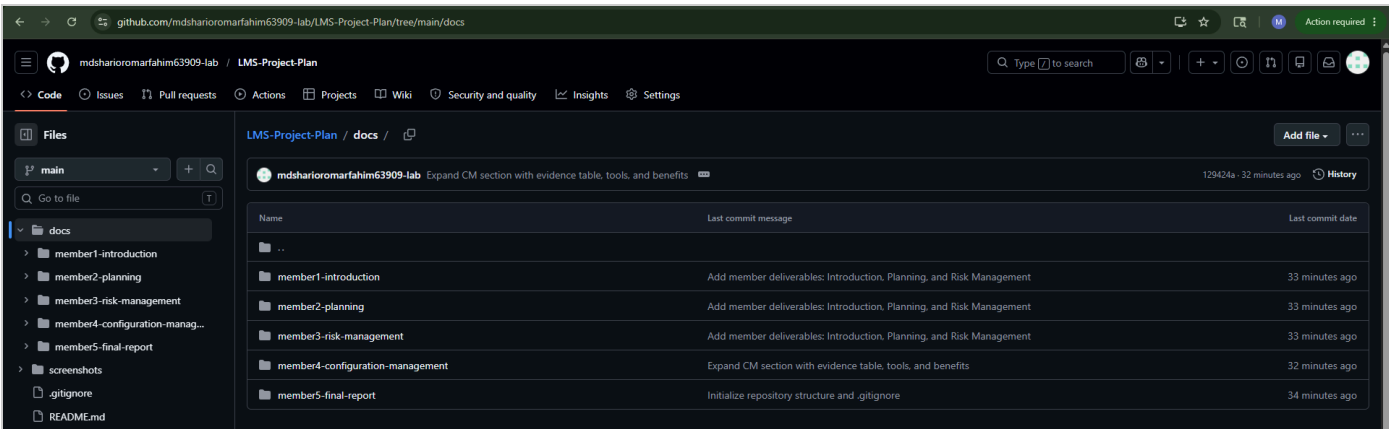


Figure 7.7 — Member files visible inside docs/

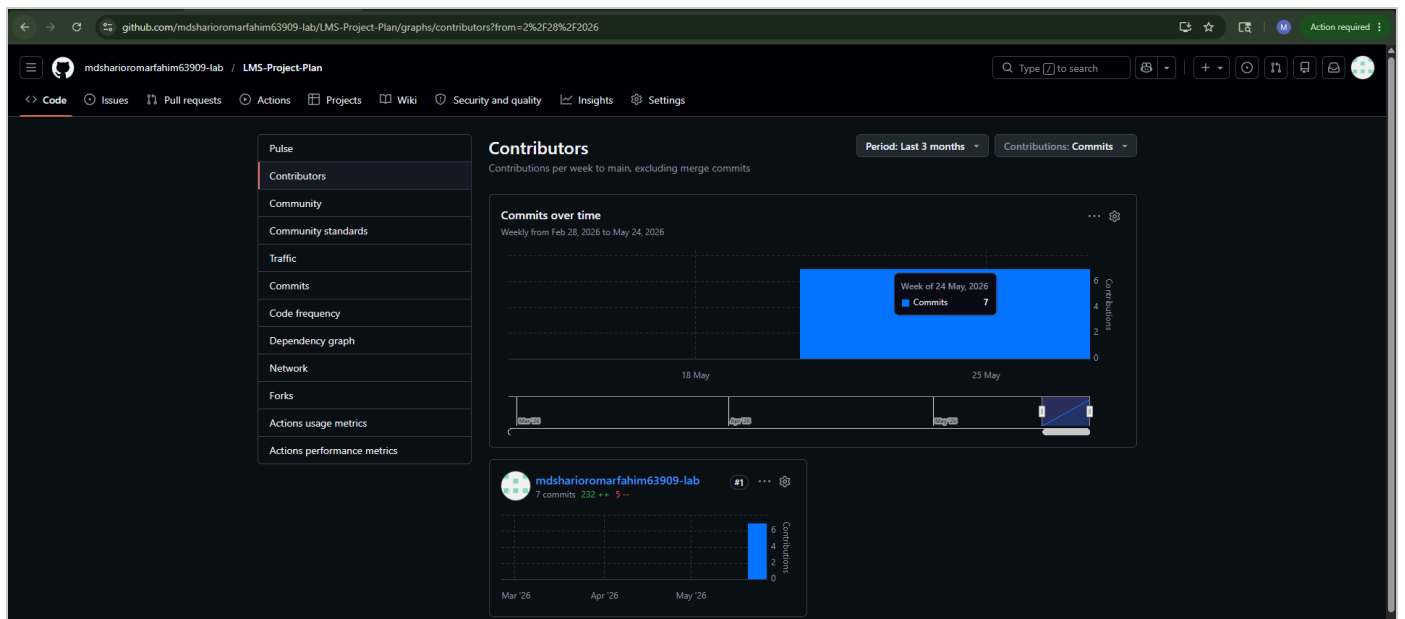


Figure 7.8 — Contributor activity